

Monitor B3 — Construindo o banco no Supabase, passo a passo

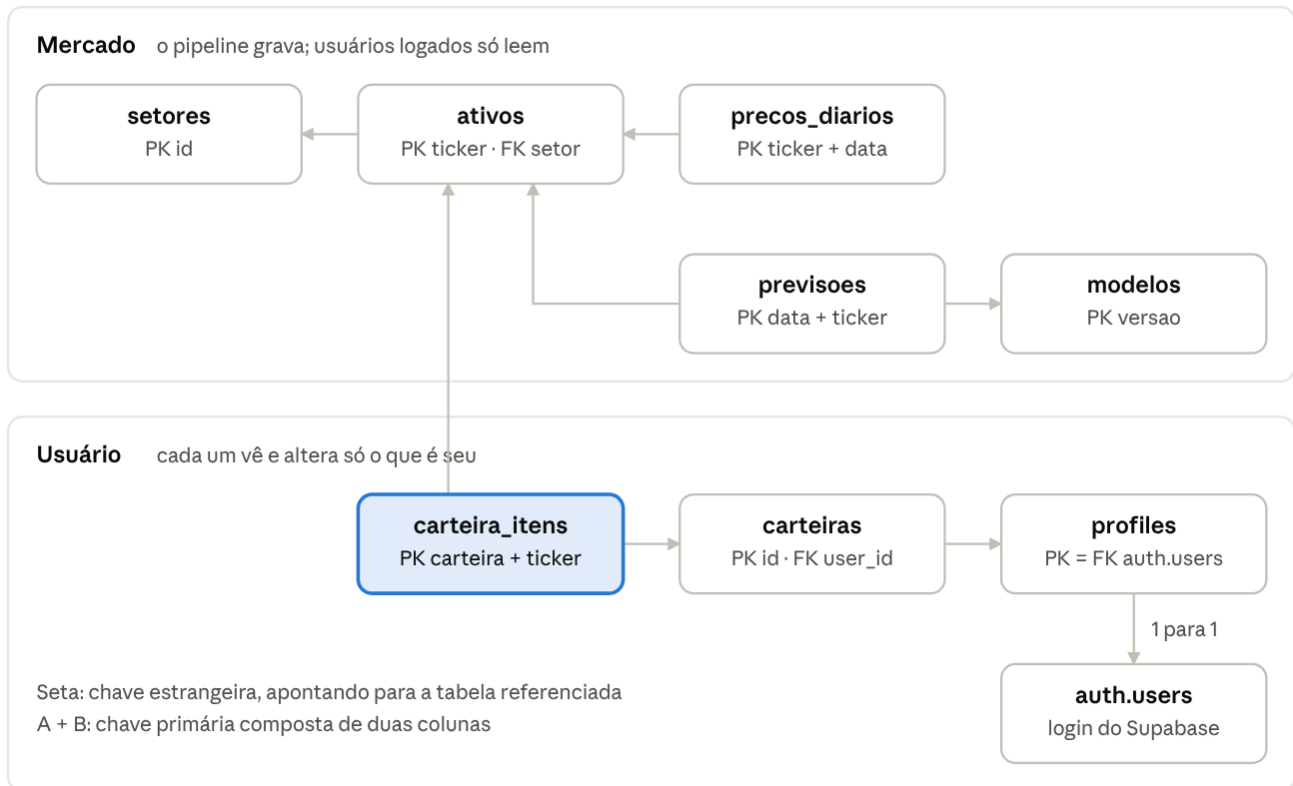
Sep 30, 2026 · @Vahid Nikoofard

O modelo de dados

O banco tem 8 tabelas em dois grupos: 5 tabelas de mercado, que o pipeline Python preenche e todos os usuários leem, e 3 tabelas do usuário, em que cada um só vê o que é seu. Juntas, elas mostram os três tipos de relação que a aula precisa cobrir.

Relação	Exemplo no projeto	Como se implementa
1 para 1	Cada usuário do login tem exatamente um perfil	A chave primária de <code>profiles</code> é também chave estrangeira para <code>auth.users</code>
1 para N	Um setor tem várias ações; uma ação tem várias cotações	Chave estrangeira no lado "N" (<code>ativos.setor_id</code> aponta para <code>setores.id</code>)
N para N	Uma carteira tem várias ações, e uma ação aparece em várias carteiras	Tabela de ligação <code>carteira_itens</code> , com duas chaves estrangeiras

Cada seta é uma chave estrangeira; carteira_itens liga carteiras e ativos



modelo de dados · 8 tabelas + auth.users

ativos é o centro do modelo: três tabelas apontam para ela. A única seta que cruza os dois grupos liga **carteira_itens** a **ativos**, porque as carteiras dos usuários são feitas de ações do mercado.

O guia constrói tudo primeiro pela interface do Supabase, sem código, para os alunos verem cada decisão. Três peças só existem em SQL (o trigger de cadastro, as views e a função de montar carteira) e entram no Passo 7. Ao final, há o script completo, que gera tudo de uma vez.

Antes de começar

Crie um projeto novo no Supabase: **New project**, nome **monitor-b3**, região **South America (São Paulo)**, plano **Free**. A criação leva 1 a 2 minutos.

A aula usa seis telas do menu lateral:

Tela	Para que serve na aula
Table Editor	Criar tabelas, colunas e relações sem código; ver e editar linhas

Tela	Para que serve na aula
SQL Editor	Rodar comandos SQL: views, funções, consultas de teste
Database → Schema Visualizer	Ver o diagrama das tabelas e das relações, gerado automaticamente
Authentication → Policies	Criar as regras de segurança (RLS) de cada tabela
Authentication → Users	Ver os usuários cadastrados pelo app
Storage	Guardar arquivos; aqui, a foto de cada usuário (Passo 6)

Dois tipos de dado aparecem o tempo todo: `int8` (inteiro grande, o mesmo que `bigint`) para identificadores numéricos e `text` para textos. Os outros são `date` (data), `numeric` (número decimal exato, para preços), `float8` (número decimal aproximado, para previsões), `bool` (verdadeiro ou falso), `uuid` (o identificador dos usuários do login) e `timestampz` (data e hora com fuso).

Passo 1 — Tabelas de mercado

As cinco tabelas são criadas no **Table Editor**, sempre na mesma ordem: primeiro as tabelas que são referenciadas, depois as que apontam para elas. Uma chave estrangeira só pode apontar para uma tabela que já existe.

Como criar cada tabela:

1. No Table Editor, clique em **New table**. Preencha o nome e mantenha marcada a opção **Enable Row Level Security (RLS)**.
2. O Supabase já sugere duas colunas: `id` (`int8`, chave primária, identidade) e `created_at`. Mantenha, altere ou apague conforme a tabela de cada passo abaixo.
3. Adicione as colunas com **Add column**. Nas opções de cada coluna (ícone de engrenagem) ficam **Is Nullable** (aceita vazio), **Is Unique** e **Is Identity** (número gerado automaticamente).
4. Para uma chave estrangeira, use a seção **Foreign keys** do painel: escolha a tabela e a coluna de destino e a ação ao apagar (**Cascade** apaga junto; o padrão impede apagar).
5. Clique em **Save**.

setores

Coluna	Tipo	Opções
<code>id</code>	<code>int8</code>	Chave primária, identidade (a sugerida)
<code>nome</code>	<code>text</code>	Não nulo, Is Unique

Apague a coluna `created_at` sugerida: ela não é necessária aqui.

ativos — relação 1 para N com **setores**

Coluna	Tipo	Opções
<code>ticker</code>	<code>text</code>	Chave primária (apague o <code>id</code> sugerido)
<code>nome</code>	<code>text</code>	Não nulo
<code>setor_id</code>	<code>int8</code>	Não nulo; chave estrangeira → <code>setores.id</code>
<code>ativo</code>	<code>bool</code>	Não nulo, valor padrão <code>true</code>

Ponto para a aula: a chave primária não precisa ser um número. O ticker já identifica a ação de forma única.

precos_diarios — chave primária composta

Coluna	Tipo	Opções
<code>ticker</code>	<code>text</code>	Chave primária; chave estrangeira → <code>ativos.ticker</code> , ao apagar Cascade
<code>data</code>	<code>date</code>	Chave primária
<code>abertura</code> , <code>maxima</code> , <code>minima</code>	<code>numeric</code>	Podem ser nulas
<code>fechamento</code>	<code>numeric</code>	Não nulo
<code>volume</code>	<code>int8</code>	Pode ser nula

Marque **Primary** nas duas colunas, `ticker` e `data`. Assim, uma mesma ação não pode ter duas cotações no mesmo dia. Se a interface não aceitar duas chaves primárias, crie essa tabela pelo trecho correspondente do script completo.

modelos

Coluna	Tipo	Opções
versao	text	Chave primária
treinado_em	timestampz	Não nulo, padrão <code>now()</code>
n_amostras	int4	Pode ser nula
ic_validacao	float8	Pode ser nula

previsoes — duas chaves estrangeiras

Coluna	Tipo	Opções
data_referencia	date	Chave primária
ticker	text	Chave primária; chave estrangeira → <code>ativos.ticker</code> , Cascade
modelo_versao	text	Não nulo; chave estrangeira → <code>modelos.versao</code> , Cascade
retorno_previsto	float8	Não nulo
posicao	int4	Não nulo
retorno_realizado	float8	Pode ser nula (preenchida 5 pregões depois)

Passo 2 — Tabelas do usuário

Essas três tabelas guardam o que é de cada pessoa. A novidade em relação ao Passo 1 é a ligação com o login do Supabase, que fica na tabela `auth.users`, criada automaticamente pelo Supabase num esquema separado.

profiles — relação 1 para 1 com `auth.users`

Coluna	Tipo	Opções
id	uuid	Chave primária (troque o tipo do <code>id</code> sugerido); chave estrangeira → <code>auth.users.id</code> , Cascade
nome	text	Pode ser nula

Coluna	Tipo	Opções
<code>avatar_url</code>	<code>text</code>	Pode ser nula (link da foto, preenchido pelo app no Passo 6)
<code>created_at</code>	<code>timestampz</code>	Não nulo, padrão <code>now()</code> (a sugerida)

Na janela da chave estrangeira, troque o esquema de `public` para `auth` para encontrar a tabela `users`. A mesma coluna é chave primária e chave estrangeira: é isso que garante "um perfil por usuário, no máximo".

`carteiras` — relação 1 para N com `profiles`

Coluna	Tipo	Opções
<code>id</code>	<code>int8</code>	Chave primária, identidade (a sugerida)
<code>user_id</code>	<code>uuid</code>	Não nulo; padrão <code>auth.uid()</code> ; chave estrangeira → <code>profiles.id</code> , Cascade
<code>nome</code>	<code>text</code>	Não nulo
<code>data_referencia</code>	<code>date</code>	Não nulo
<code>created_at</code>	<code>timestampz</code>	Não nulo, padrão <code>now()</code> (a sugerida)

O valor padrão `auth.uid()` é uma função do Supabase que devolve o id de quem está logado. Com ela, o frontend nunca precisa enviar o `user_id`: o banco preenche sozinho, e o usuário não consegue se passar por outro.

`carteira_itens` — a tabela de ligação N para N

Coluna	Tipo	Opções
<code>carteira_id</code>	<code>int8</code>	Chave primária; chave estrangeira → <code>carteiras.id</code> , Cascade
<code>ticker</code>	<code>text</code>	Chave primária; chave estrangeira → <code>ativos.ticker</code>
<code>peso</code>	<code>numeric</code>	Não nulo

Apague o `id` e o `created_at` sugeridos. A chave primária composta (`carteira_id`, `ticker`) impede a mesma ação de entrar duas vezes na mesma carteira. O script completo acrescenta uma regra que a interface não cobre facilmente: o peso precisa ficar

entre 0 e 1.

Passo 3 — Dados iniciais

O pipeline Python lê da tabela `ativos` quais ações deve acompanhar. Por isso, setores e ações são cadastrados à mão, uma única vez. Para incluir uma ação nova depois, basta inserir uma linha: o pipeline passa a acompanhá-la no dia seguinte, sem mudar código.

Pela interface (para mostrar uma linha): no Table Editor, abra `setores`, clique em **Insert** → **Insert row** e preencha só o `nome`, por exemplo `Bancos`; o `id` é gerado sozinho. Depois, em `ativos`, insira `ITUB4`, `Itaú Unibanco PN` e, em `setor_id`, escolha o setor na lista: a interface mostra as opções porque reconhece a chave estrangeira.

Pelo SQL Editor (para o resto): cadastrar 24 ações clicando é lento. O `INSERT` abaixo faz tudo de uma vez e ainda é um bom exemplo de `INSERT ... SELECT` com `JOIN`, que busca o `id` de cada setor pelo nome.

```
insert into setores (nome) values
  ('Petróleo e gás'), ('Mineração'), ('Siderurgia'), ('Papel e celulose'),
  ('Bancos'), ('Serviços financeiros'), ('Energia elétrica'), ('Saneamento'),
  ('Telecom'), ('Tecnologia'), ('Bens de capital'), ('Locação'),
  ('Bebidas'), ('Saúde'), ('Varejo'), ('Construção')
on conflict (nome) do nothing;

insert into ativos (ticker, nome, setor_id)
select a.ticker, a.nome, s.id
from (values
  ('PETR4', 'Petrobras PN', 'Petróleo e gás'), ('PRI03', 'PRIO', 'Petróleo
e gás'),
  ('VALE3', 'Vale', 'Mineração'), ('GGBR4', 'Gerdau PN',
'Siderurgia'),
  ('SUZB3', 'Suzano', 'Papel e celulose'), ('KLBN11', 'Klabin Unit',
'Papel e celulose'),
  ('ITUB4', 'Itaú Unibanco PN', 'Bancos'), ('BBDC4', 'Bradesco PN',
'Bancos'),
  ('BBAS3', 'Banco do Brasil', 'Bancos'), ('BPAC11', 'BTG Pactual
Unit', 'Bancos'),
  ('ITSA4', 'Itaúsa PN', 'Bancos'), ('B3SA3', 'B3', 'Serviços
financeiros'),
  ('EQTL3', 'Equatorial', 'Energia elétrica'), ('CMIG4', 'Cemig PN',
'Energia elétrica'),
  ('SBSP3', 'Sabesp', 'Saneamento'), ('VIVT3', 'Telefônica
Brasil', 'Telecom'),
  ('TOTS3', 'Totvs', 'Tecnologia'), ('WEGE3', 'WEG', 'Bens de
capital'),
```

```

('RENT3', 'Localiza', 'Locação'),
'Bebidas'),
('RADL3', 'Raia Drogasil', 'Saúde'),
'Varejo'),
('MGLU3', 'Magazine Luiza', 'Varejo'),
'Construção')
) as a(ticker, nome, setor)
join setores s on s.nome = a.setor
on conflict (ticker) do nothing;
```

O `on conflict ... do nothing` ignora as linhas que já existem, como `Bancos` e `ITUB4`, se você as tiver inserido pela interface antes.

Passo 4 — Ver as relações no Schema Visualizer

Abra **Database** → **Schema Visualizer**. O Supabase desenha uma caixa por tabela e uma linha por chave estrangeira, a partir do que foi criado nos Passos 1 e 2. É o melhor momento para conferir o trabalho com a turma.

- ☐ As 8 tabelas aparecem, cada uma com suas colunas e tipos
- ☐ `ativos` tem linhas saindo para `precos_diarios`, `previsoes` e `carteira_itens`, e uma chegando de `setores`
- ☐ `carteira_itens` recebe duas linhas, uma de `carteiras` e outra de `ativos`: é o desenho de uma relação N para N
- ☐ As chaves primárias aparecem marcadas; em `precos_diarios`, `previsoes` e `carteira_itens` há duas colunas marcadas (chaves compostas)

Se uma linha estiver faltando, a chave estrangeira daquela coluna não foi criada. Volte ao Table Editor, abra a tabela pelo menu **Edit table** e adicione a relação.

Passo 5 — Segurança: RLS e políticas

Com a RLS ligada e nenhuma política criada, ninguém acessa nada pelo app: a tabela fica trancada. Cada política abre uma porta específica. O app tem só dois tipos de porta: "qualquer logado lê" para o mercado e "só o dono" para as carteiras.

Abra **Authentication** → **Policies**. Cada tabela aparece com o aviso de RLS ativa. Em cada uma, clique em **Create policy** e preencha:

Tabela	Nome da política	Comando	Papel (role)	Condição (USING)
<code>setores</code> , <code>ativos</code> , <code>precos_diarios</code> , <code>modelos</code> , <code>previsoes</code>	leitura para logados	SELECT	<code>authenticated</code>	<code>true</code>
<code>profiles</code>	ver o próprio perfil	SELECT	<code>authenticated</code>	<code>id = auth.uid()</code>
<code>profiles</code>	editar o próprio perfil	UPDATE	<code>authenticated</code>	<code>id = auth.uid()</code>
<code>carteiras</code>	dono da carteira	ALL	<code>authenticated</code>	<code>user_id = auth.uid()</code>
<code>carteira_itens</code>	dono da carteira	ALL	<code>authenticated</code>	<code>exists (select 1 from carteiras c where c.id = carteira_id and c.user_id = auth.uid())</code>

Nas políticas de UPDATE e ALL, repita a mesma condição no campo **WITH CHECK**. O USING decide quais linhas a pessoa enxerga; o WITH CHECK decide o que ela pode gravar.

Três pontos para discutir com a turma:

- As tabelas de mercado não têm política de escrita. Quem grava nelas é o pipeline Python, com a chave secreta, que ignora a RLS.
- `carteira_itens` não tem `user_id`. A política descobre o dono olhando a carteira a que o item pertence: é uma subconsulta dentro da regra de segurança.
- O editor de políticas mostra, embaixo, o comando SQL equivalente. Vale mostrar aos alunos que a interface está só escrevendo SQL por eles.

Passo 6 — Storage: a foto de cada usuário

O Storage guarda os arquivos; o banco guarda só o endereço. A foto de cada usuário fica no bucket `avatares`, numa pasta com o id dele (`<id do usuário>/avatar`), e o link vai para a coluna `profiles.avatar_url`.

Criar o bucket pela interface:

1. No menu lateral, abra **Storage** e clique em **New bucket**.
2. Nome: `avatares` . Ligue a opção **Public bucket**.
3. Nas opções adicionais, limite o tamanho do arquivo a **2 MB** e os tipos permitidos a `image/png` , `image/jpeg` e `image/webp` .
4. Clique em **Save**.

Bucket público significa que quem tiver o link consegue ver a foto, o que é adequado para avatar. Enviar, trocar e apagar continua controlado pelas políticas abaixo. Para arquivos sigilosos, como documentos, use um bucket privado.

Criar as políticas: no Storage, abra a aba de políticas, escolha o bucket `avatares` e crie quatro políticas, todas para o papel `authenticated` :

Política	Comando	Condição
logados veem	SELECT	<code>bucket_id = 'avatares'</code>
enviar na própria pasta	INSERT	<code>bucket_id = 'avatares' and (storage.foldername(name))[1] = auth.uid()::text</code>
trocar na própria pasta	UPDATE	a mesma condição do INSERT, em USING e em WITH CHECK
apagar da própria pasta	DELETE	a mesma condição do INSERT

A função `storage.foldername(name)` quebra o caminho do arquivo em pastas; `[1]` é a primeira. A regra exige que a primeira pasta seja o id de quem está logado: cada um só mexe na própria foto. É a mesma ideia do "só o dono" do Passo 5, aplicada a arquivos.

O mesmo, em SQL:

```

insert into storage.buckets (id, name, public, file_size_limit,
allowed_mime_types)
values ('avatares', 'avatares', true, 2097152,
      array['image/png', 'image/jpeg', 'image/webp'])
on conflict (id) do nothing;

create policy "avatares: logados veem" on storage.objects for select to
authenticated
  using (bucket_id = 'avatares');

create policy "avatares: enviar na própria pasta" on storage.objects for
insert to authenticated
  with check (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

create policy "avatares: trocar na própria pasta" on storage.objects for
update to authenticated
  using (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text)
  with check (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

create policy "avatares: apagar da própria pasta" on storage.objects for
delete to authenticated
  using (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

```

Se você criou a tabela `profiles` antes desta versão do guia, acrescente a coluna que guarda o link: `alter table public.profiles add column avatar_url text;`

Passo 7 — O que só se faz com SQL

Três peças não têm tela no Table Editor: o trigger que cria o perfil, as views e a função que monta a carteira. Cole cada bloco no **SQL Editor** e clique em **Run**.

Trigger: todo usuário novo ganha um perfil

Quando alguém se cadastra no app, o Supabase insere uma linha em `auth.users`. O trigger reage a esse evento e cria a linha correspondente em `profiles`.

```

create or replace function public.criar_perfil()
returns trigger language plpgsql security definer set search_path = ''
as $$
begin
    insert into public.profiles (id, nome)
    values (new.id, new.raw_user_meta_data ->> 'nome');
    return new;
end;
$$;

create trigger ao_criar_usuario
after insert on auth.users
for each row execute function public.criar_perfil();

```

Views: consultas prontas para o frontend

Uma view é uma consulta com nome. O frontend lê `v_ranking_atual` como se fosse uma tabela, sem precisar saber fazer os `JOIN`s. A opção `security_invoker = true` é obrigatória: sem ela, a view ignoraria a RLS e mostraria as carteiras de todos os usuários.

```

create view public.v_ranking_atual with (security_invoker = true) as
select p.posicao, p.ticker, a.nome, s.nome as setor,
       p.retorno_previsto, pr.fechamento as ultimo_preco,
       p.data_referencia, p.modelo_versao
from public.previsoes p
join public.ativos a on a.ticker = p.ticker
join public.setores s on s.id = a.setor_id
left join public.precos_diarios pr on pr.ticker = p.ticker and pr.data =
p.data_referencia
where p.data_referencia = (select max(data_referencia) from
public.previsoes);

create view public.v_desempenho_carteiras with (security_invoker = true) as
select c.id as carteira_id, c.user_id, c.nome, c.data_referencia,
       count(*) as n_acoes,
       sum(i.peso * p.retorno_previsto) as retorno_previsto,
       sum(i.peso * p.retorno_realizado) as retorno_realizado,
       count(p.retorno_realizado) = count(*) as concluida
from public.carteiras c
join public.carteira_itens i on i.carteira_id = c.id
left join public.previsoes p on p.ticker = i.ticker and p.data_referencia =
c.data_referencia
group by c.id;

```

Função: montar a carteira

O botão "Gerar carteira" do app chama esta função. Ela pega as N ações com melhor previsão no último pregão, cria a carteira e os itens com pesos iguais, e devolve o id da carteira. A regra fica no banco, não no frontend.

```
create or replace function public.montar_carteira(p_n integer default 5,
p_nome text default null)
returns bigint
language plpgsql security invoker set search_path = public
as $$
declare
    v_data date;
    v_id    bigint;
begin
    if auth.uid() is null then
        raise exception 'É preciso estar logado';
    end if;
    if p_n not between 1 and 20 then
        raise exception 'Escolha entre 1 e 20 ações';
    end if;

    select max(data_referencia) into v_data from previsoes;
    if v_data is null then
        raise exception 'Ainda não há previsões';
    end if;

    insert into carteiras (nome, data_referencia)
    values (coalesce(p_nome, 'Carteira de ' || to_char(v_data, 'DD/MM/YYYY')),
v_data)
    returning id into v_id;

    insert into carteira_itens (carteira_id, ticker, peso)
    select v_id, t.ticker, round(1.0 / count(*) over (), 4)
    from (select ticker from previsoes
        where data_referencia = v_data
        order by posicao
        limit p_n) t;

    return v_id;
end;
$$;
```

```
revoke execute on function public.montar_carteira(integer, text) from anon,
public;
grant execute on function public.montar_carteira(integer, text) to
authenticated;
```

A função roda com as permissões de quem a chama (`security invoker`), então as políticas do Passo 5 continuam valendo: ela só consegue criar carteiras para o usuário logado.

Passo 8 — Testar com consultas SQL

As consultas abaixo conferem o banco e servem como exercícios de SQL. As do primeiro bloco já funcionam agora; as do segundo, depois que o pipeline Python rodar pela primeira vez.

Já funcionam: setores e ações

```
-- Quantas ações por setor (JOIN + GROUP BY)
select s.nome as setor, count(a.ticker) as acoes
from setores s
left join ativos a on a.setor_id = s.id
group by s.nome
order by acoes desc, setor;

-- A chave estrangeira em ação: esta inserção deve FALHAR (setor 999 não
existe)
insert into ativos (ticker, nome, setor_id) values ('TESTE3', 'Teste', 999);

-- A função exige login: no SQL Editor não há usuário logado, então deve
FALHAR
select montar_carteira(5);
```

Os dois erros são o resultado esperado e mostram as regras do banco funcionando: a chave estrangeira recusa um setor inexistente, e a função recusa quem não está logado.

Depois da primeira execução do pipeline

```
-- Último fechamento de cada ação, com o setor (JOIN em três tabelas)
select a.ticker, s.nome as setor, p.data, p.fechamento
from precos_diarios p
join ativos a on a.ticker = p.ticker
join setores s on s.id = a.setor_id
where p.data = (select max(data) from precos_diarios)
order by a.ticker;

-- Ranking do dia, pela view
select posicao, ticker, setor, round(retorno_previsto::numeric, 4) as
previsto
from v_ranking_atual
order by posicao
limit 10;

-- Histórico do modelo: uma linha por treino diário
select versao, treinado_em, n_amstras, ic_validacao
from modelos
order by treinado_em desc;
```

O teste das carteiras é feito pelo app, com dois usuários diferentes: o usuário B não deve ver as carteiras do usuário A. É a demonstração prática das políticas do Passo 5.

Script SQL completo

Este script cria tudo dos Passos 1 a 7 de uma vez, num projeto vazio: tabelas, relações, dados iniciais, bucket de fotos, trigger, views, função e políticas de segurança. Cole no **SQL Editor** e clique em **Run**, uma única vez. Ele também está no arquivo `sql/01_schema.sql` do projeto.

```
-- =====
-- MONITOR B3 (versão de aula) – schema completo do Supabase
-- Execute no SQL Editor, uma única vez, de cima para baixo.
--
-- 8 tabelas:
-- MERCADO (compartilhadas, só leitura para o app):
--   setores —< ativos —< precos_diarios
--           └─< previsoes >— modelos
-- USUÁRIO (privadas, cada um vê só o que é seu):
--   auth.users — profiles —< carteiras —< carteira_itens >— ativos
-- STORAGE: bucket 'avatares' com a foto de cada usuário (link em
-- profiles.avatar_url)
-- =====
```

```

-- -----
-- 1. TABELAS DE MERCADO
-- -----

-- Setores da economia (1 setor tem N ações)
create table public.setores (
  id   bigint generated by default as identity primary key,
  nome text not null unique
);

-- Ações acompanhadas pelo app (N ações pertencem a 1 setor)
create table public.ativos (
  ticker   text primary key,          -- ex.: 'PETR4' (sem o
  .SA)
  nome      text not null,
  setor_id  bigint not null references public.setores(id),
  ativo     boolean not null default true -- false = o pipeline
ignora
);

-- Cotações diárias (1 ação tem N cotações; chave = ação + data)
create table public.precos_diarios (
  ticker   text not null references public.ativos(ticker) on delete
cascade,
  data      date not null,
  abertura  numeric(12,4),
  maxima    numeric(12,4),
  minima    numeric(12,4),
  fechamento numeric(12,4) not null,
  volume     bigint,
  primary key (ticker, data)
);

-- Cada treino diário do modelo gera uma versão
create table public.modelos (
  versao     text primary key,          -- ex.: '2026-09-30T22-
05'
  treinado_em timestampz not null default now(),
  n_amostras integer,
  ic_validacao double precision        -- qualidade medida na
validação
);

-- Previsão de retorno para os próximos 5 pregões, uma por ação e por dia

```

```

create table public.previsoes (
  data_referencia date not null,
  ticker          text not null references public.ativos(ticker) on delete
cascade,
  modelo-versao   text not null references public.modelos(versao) on delete
cascade,
  retorno_previsto double precision not null,
  posicao          integer not null,          -- 1 = maior retorno
previsto
  retorno_realizado double precision,        -- preenchido 5 pregões
depois
  primary key (data_referencia, ticker)
);

-- -----
-- 2. TABELAS DO USUÁRIO
-- -----

-- Perfil: 1 para 1 com o usuário do Supabase Auth
create table public.profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  nome        text,
  avatar_url  text,          -- link da foto no
Storage (bucket 'avatares')
  created_at  timestamptz not null default now()
);

-- Carteiras montadas pelo usuário (1 usuário tem N carteiras)
create table public.carteiras (
  id          bigint generated by default as identity primary key,
  user_id     uuid not null default auth.uid() references
public.profiles(id) on delete cascade,
  nome        text not null,
  data_referencia date not null,
  created_at  timestamptz not null default now()
);

-- Ações de cada carteira: liga carteiras e ativos (relação N para N)
create table public.carteira_itens (
  carteira_id bigint not null references public.carteiras(id) on delete
cascade,
  ticker      text not null references public.ativos(ticker),
  peso        numeric(5,4) not null check (peso > 0 and peso <= 1),
  primary key (carteira_id, ticker)
);

```

```
-- -----
-- 3. DADOS INICIAIS: setores e ações
-- -----
```

```
insert into public.setores (nome) values
  ('Petróleo e gás'), ('Mineração'), ('Siderurgia'), ('Papel e celulose'),
  ('Bancos'), ('Serviços financeiros'), ('Energia elétrica'), ('Saneamento'),
  ('Telecom'), ('Tecnologia'), ('Bens de capital'), ('Locação'),
  ('Bebidas'), ('Saúde'), ('Varejo'), ('Construção')
on conflict (nome) do nothing;
```

```
insert into public.ativos (ticker, nome, setor_id)
select a.ticker, a.nome, s.id
from (values
  ('PETR4', 'Petrobras PN', 'Petróleo e gás'),
  ('PRIO3', 'PRIO', 'Petróleo e gás'),
  ('VALE3', 'Vale', 'Mineração'),
  ('GGBR4', 'Gerdau PN', 'Siderurgia'),
  ('SUZB3', 'Suzano', 'Papel e celulose'),
  ('KLBN11', 'Klabin Unit', 'Papel e celulose'),
  ('ITUB4', 'Itaú Unibanco PN', 'Bancos'),
  ('BBDC4', 'Bradesco PN', 'Bancos'),
  ('BBAS3', 'Banco do Brasil', 'Bancos'),
  ('BPAC11', 'BTG Pactual Unit', 'Bancos'),
  ('ITSA4', 'Itaúsa PN', 'Bancos'),
  ('B3SA3', 'B3', 'Serviços financeiros'),
  ('EQL3', 'Equatorial', 'Energia elétrica'),
  ('CMIG4', 'Cemig PN', 'Energia elétrica'),
  ('SBSP3', 'Sabesp', 'Saneamento'),
  ('VIVT3', 'Telefônica Brasil', 'Telecom'),
  ('TOTS3', 'Totvs', 'Tecnologia'),
  ('WEGE3', 'WEG', 'Bens de capital'),
  ('RENT3', 'Localiza', 'Locação'),
  ('ABEV3', 'Ambev', 'Bebidas'),
  ('RADL3', 'Raia Drogasil', 'Saúde'),
  ('LREN3', 'Lojas Renner', 'Varejo'),
  ('MGLU3', 'Magazine Luiza', 'Varejo'),
  ('CYRE3', 'Cyrela', 'Construção')
) as a(ticker, nome, setor)
join public.setores s on s.nome = a.setor
on conflict (ticker) do nothing;
```

```
-- -----
-- 4. TRIGGER: todo usuário novo ganha uma linha em profiles
-- -----
```

```

-- -----
create or replace function public.criar_perfil()
returns trigger language plpgsql security definer set search_path = ''
as $$
begin
    insert into public.profiles (id, nome)
    values (new.id, new.raw_user_meta_data ->> 'nome');
    return new;
end;
$$;

create trigger ao_criar_usuario
after insert on auth.users
for each row execute function public.criar_perfil();

-- -----

-- 5. VIEWS para o frontend
-- security_invoker = true: a view respeita a RLS de quem consulta
-- -----

-- Ranking do último pregão, com nome, setor e último preço
create view public.v_ranking_atual with (security_invoker = true) as
select p.posicao, p.ticker, a.nome, s.nome as setor,
       p.retorno_previsto, pr.fechamento as ultimo_preco,
       p.data_referencia, p.modelo_versao
from public.previsoes p
join public.ativos a on a.ticker = p.ticker
join public.setores s on s.id = a.setor_id
left join public.precos_diarios pr on pr.ticker = p.ticker and pr.data =
p.data_referencia
where p.data_referencia = (select max(data_referencia) from
public.previsoes);

-- Resultado de cada carteira do usuário: previsto × realizado
create view public.v_desempenho_carteiras with (security_invoker = true) as
select c.id as carteira_id, c.user_id, c.nome, c.data_referencia,
       count(*) as n_acoes,
       sum(i.peso * p.retorno_previsto) as retorno_previsto,
       sum(i.peso * p.retorno_realizado) as retorno_realizado,
       count(p.retorno_realizado) = count(*) as concluida
from public.carteiras c
join public.carteira_itens i on i.carteira_id = c.id
left join public.previsoes p on p.ticker = i.ticker and p.data_referencia =
c.data_referencia
group by c.id;

```

```

-- -----
-- 6. FUNÇÃO chamada pelo frontend: monta uma carteira com as N melhores
--    previsões do último pregão, com pesos iguais. Retorna o id criado.
-- -----

create or replace function public.montar_carteira(p_n integer default 5,
p_nome text default null)
returns bigint
language plpgsql security invoker set search_path = public
as $$
declare
    v_data date;
    v_id    bigint;
begin
    if auth.uid() is null then
        raise exception 'É preciso estar logado';
    end if;
    if p_n not between 1 and 20 then
        raise exception 'Escolha entre 1 e 20 ações';
    end if;

    select max(data_referencia) into v_data from previsoes;
    if v_data is null then
        raise exception 'Ainda não há previsões';
    end if;

    insert into carteiras (nome, data_referencia)
    values (coalesce(p_nome, 'Carteira de ' || to_char(v_data, 'DD/MM/YYYY')),
v_data)
    returning id into v_id;

    insert into carteira_itens (carteira_id, ticker, peso)
    select v_id, t.ticker, round(1.0 / count(*) over (), 4)
    from (select ticker from previsoes
        where data_referencia = v_data
        order by posicao
        limit p_n) t;

    return v_id;
end;
$$;

-- -----
-- 7. SEGURANÇA — Row Level Security

```

```

-- -----
alter table public.setores          enable row level security;
alter table public.ativos           enable row level security;
alter table public.precos_diarios   enable row level security;
alter table public.modelos          enable row level security;
alter table public.previsoes        enable row level security;
alter table public.profiles         enable row level security;
alter table public.carteiras        enable row level security;
alter table public.carteira_itens   enable row level security;

-- Mercado: qualquer usuário logado lê; ninguém escreve pelo app
-- (o pipeline Python escreve com a chave secreta, que ignora a RLS)
create policy "leitura para logados" on public.setores          for select to
authenticated using (true);
create policy "leitura para logados" on public.ativos           for select to
authenticated using (true);
create policy "leitura para logados" on public.precos_diarios   for select to
authenticated using (true);
create policy "leitura para logados" on public.modelos          for select to
authenticated using (true);
create policy "leitura para logados" on public.previsoes        for select to
authenticated using (true);

-- Usuário: cada um só vê e altera o que é seu
create policy "ver o próprio perfil" on public.profiles for select to
authenticated
    using (id = auth.uid());
create policy "editar o próprio perfil" on public.profiles for update to
authenticated
    using (id = auth.uid()) with check (id = auth.uid());

create policy "dono da carteira" on public.carteiras for all to authenticated
    using (user_id = auth.uid()) with check (user_id = auth.uid());

create policy "dono da carteira" on public.carteira_itens for all to
authenticated
    using (exists (select 1 from public.carteiras c
                    where c.id = carteira_id and c.user_id = auth.uid()))
    with check (exists (select 1 from public.carteiras c
                        where c.id = carteira_id and c.user_id = auth.uid()));

-- -----
-- 8. STORAGE — foto de cada usuário
-- Arquivo em avatares/<id do usuário>/avatar; cada um só mexe na própria
pasta.
-- Bucket público: quem tiver o link vê a foto (adequado para avatar).

```

```

-- -----
insert into storage.buckets (id, name, public, file_size_limit,
allowed_mime_types)
values ('avatares', 'avatares', true, 2097152,          -- até 2 MB
        array['image/png', 'image/jpeg', 'image/webp'])
on conflict (id) do nothing;

create policy "avatares: logados veem" on storage.objects for select to
authenticated
    using (bucket_id = 'avatares');

create policy "avatares: enviar na própria pasta" on storage.objects for
insert to authenticated
    with check (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

create policy "avatares: trocar na própria pasta" on storage.objects for
update to authenticated
    using (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text)
    with check (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

create policy "avatares: apagar da própria pasta" on storage.objects for
delete to authenticated
    using (bucket_id = 'avatares' and (storage.foldername(name))[1] =
auth.uid()::text);

-- Visitantes não logados não acessam nada
revoke all on all tables in schema public from anon;
revoke execute on function public.montar_carteira(integer, text) from anon,
public;
grant execute on function public.montar_carteira(integer, text) to
authenticated;

```